

GEOquery

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Overview](#)
- [Other Resources](#)
- [NCBI GEO](#)
- [GEOquery](#)
- [Other packages](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(GEOquery)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GEOquery"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Overview

NCBI Gene Expression Omnibus (GEO) has a lot of high-throughput genomics datasets publicly available. Despite the name, this database is not exclusively for gene expression data.

Other Resources

- The vignette from the [GEOquery webpage](#).
- [GEO documentation](#).

NCBI GEO

NCBI GEO is organised as samples which are grouped into series. For bigger experiments there are both SubSeries and SuperSeries. A SuperSeries is all the experiments for a single paper; a SuperSeries can be decomposed into SubSeries which are different technologies. As an example, look at the SuperSeries [GSE19486](#). In this paper they used 2 different platforms (this is a weird name; a platform is a combination of a technology and a species). And they did RNA-seq and ChIP-seq for two different factors (NFkB-II and Pol II). This results in 4 SubSeries (2 for RNA-seq and 2 for ChIP-seq).

A simpler setup is for example [GSE994](#) where samples from current and former smokers were compared, using an Affymetrix microarray.

Data submitted to NCBI GEO can be both “raw” and “processed”. Let us focus on gene expression data for the moment. “Processed” data is normalized and quantified, typically at the gene level, and is usually provided in the form of a gene by sample matrix. “Raw” data can be anything, from sequencing reads to microarray image files. There can even be different states of “raw” data, for example for a RNA-seq dataset you may have

- FASTQ files (raw reads)
- BAM files (aligned reads)
- gene by sample expression matrix (unnormalized)
- gene by sample expression matrix (normalized)

So what is “raw” and what is “processed” can be very context dependent.

In some cases there is a consensus in the field. For Affymetrix gene expression microarrays, “raw” files are so-called CEL files (a file format invented by Affymetrix) and “processed” data is normalized and quantified data, summarized at the probeset level.

At the end of the day, GEO has series identifiers (like [GSE19486](#)) and sample identifiers ([GSM486297](#)). Note the GSE vs GSM in the same. A user is almost always interested in all the samples in a given series, so the starting point is the series identifier, also called the accession number.

GEOquery

All you need to download data from GEO is the accession number. Let us use [GSE11675](#) which is a very small scale Affymetrix gene expression array study (6 samples).

```
eList <- getGEO("GSE11675")
class(eList)
```

```
## [1] "list"
```

```
length(eList)
```

```
## [1] 1
```

```
names(eList)
```

```
## [1] "GSE11675_series_matrix.txt.gz"
```

```
eData <- eList[[1]]
eData
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 12625 features, 6 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: GSM296630 GSM296635 ... GSM296639 (6 total)
##   varLabels: title geo_accession ... data_row_count (34 total)
##   varMetadata: labelDescription
## featureData
##   featureNames: 1000_at 1001_at ... AFFX-YEL024w/RIP1_at (12625
##     total)
##   fvarLabels: ID GB_ACC ... Gene Ontology Molecular Function (16
##     total)
##   fvarMetadata: Column Description labelDescription
## experimentData: use 'experimentData(object)'
## Annotation: GPL8300
```

The function returns a list because you might be getting multiple SubSeries. In this case there is only one, and the list element is an ExpressionSet ready for usage! The phenotype data (which GEO knows about) is contained inside the pData slot; there is usually a lot of

unnecessary stuff here:

```
names(pData(eData))
```

```
## [1] "title" "geo_accession"
## [3] "status" "submission_date"
## [5] "last_update_date" "type"
## [7] "channel_count" "source_name_ch1"
## [9] "organism_ch1" "characteristics_ch1"
## [11] "characteristics_ch1.1" "characteristics_ch1.2"
## [13] "molecule_ch1" "extract_protocol_ch1"
## [15] "label_ch1" "label_protocol_ch1"
## [17] "taxid_ch1" "hyb_protocol"
## [19] "scan_protocol" "description"
## [21] "data_processing" "platform_id"
## [23] "contact_name" "contact_email"
## [25] "contact_phone" "contact_department"
## [27] "contact_institute" "contact_address"
## [29] "contact_city" "contact_zip/postal_code"
## [31] "contact_country" "supplementary_file"
## [33] "supplementary_file.1" "data_row_count"
```

However, what we got here was processed data. Users (including me) often want access to more raw data. This is called “supplementary files” in GEO language and we can get those as well.

```
eList2 <- getGEOSuppFiles("GSE11675")
eList2
```

```
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar 458
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar FAL
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt FAL
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar 64
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt 64
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar 201
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt 201
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar 201
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt 201
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar 201
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt 201
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar 501
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt 501
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar kha
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt kha
##
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.tar st
## /Users/khansen/work/genbioconductor/Rmd/GSE11675/filelist.txt st
```

```
tarArchive <- rownames(eList2)[1]
tarArchive
```

```
## [1] "/Users/khansen/work/genbioconductor/Rmd/GSE11675/GSE11675_RAW.ta
```

This is now a data.frame of file names. A single TAR archive was downloaded. You can expand the TAR archive using standard tools; inside there is a list of 6 CEL files and 6 CHP files. You can then read the 6 CEL files into R using functions from [affy](#) or [oligo](#).

It is also possible to use [GEOquery](#) to query GEO as a database (ie. looking for datasets); more information in the package vignette.

Other packages

There are other packages for accessing other online repositories with public data; they include [SRADB](#) for the Short Read Archive (SRA) and [ArrayExpress](#) (ArrayExpress; a similar database to NCBI GEO but hosted at the European Bioinformatics Institute (EBI)).

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] parallel  methods    stats      graphics  grDevices  utils      datas
## [8] base
##
## other attached packages:
## [1] GEOquery_2.34.0      Biobase_2.28.0      BiocGenerics_0.14.0
## [4] BiocStyle_1.6.0      rmarkdown_0.7
##
## loaded via a namespace (and not attached):
## [1] XML_3.98-1.3    digest_0.6.8    bitops_1.0-6    formatR_1.2
## [5] magrittr_1.5    evaluate_0.7.2  stringi_0.5-5    tools_3.2.1
## [9] stringr_1.0.0   RCurl_1.95-4.7  yaml_2.1.13     htmltools_0.2.6
## [13] knitr_1.11
```

